

Проектирование больших систем на C++

Жуков Владимир Владимирович

E-mail: zhvv117@gmail.com

Telegram: @zhvv117

Лекция 1

- Курс посвящен проектированию и программированию больших систем на языке C++
- Основные темы курса:
 - Особенности C++11/14/17/20/23
 - Вывод типов для шаблонов и auto
 - Семантика перемещения
 - Умные указатели и владение памятью
 - Паттерны и идиомы проектирования
 - Метапрограммирование, правило SFINAE и его применение
 - Обработка ошибок, тестирование и отладка кода
 - Оптимизация программ на C++

- Лекции: еженедельно
- Экзамен в конце курса (в т. ч. досрочный в декабре)
- Материалы курса доступны на сайте **mk.cs.msu.ru** (Лекционные курсы
→ Проектирование больших систем на C++)
- Вопросы можно задавать в Telegram (будет создан чат) или по электронной почте zhvv117@gmail.com
- От старост групп — **списки студентов**

Вводный тест

- C++ — мультипарадигмальный язык программирования
 - Поддерживает процедурное программирование
 - Объектно-ориентированное программирование
 - Обобщённое программирование (шаблоны)
 - Функциональное программирование (с ограничениями)
- Сочетает высокоуровневые абстракции и низкоуровневый контроль
- Обеспечивает высокую производительность
- Широко используется в:
 - Системном программировании
 - Разработке игр
 - Высоконагруженных серверных приложениях
 - Встраиваемых системах
 - Научных вычислениях

- C++ создан Бьёрном Страуструпом в начале 1980-х
- Стандарты языка:
 - C++98/03 — первый стандарт
 - C++11 — значительное обновление (auto, lambda, move semantics)
 - C++14 — улучшения C++11
 - C++17 — новые возможности (structured bindings, std::optional)
 - C++20 — concepts, coroutines, ranges
 - C++23 — улучшения C++20
 - C++26 — статическая рефлексия (операторы «баян» ([:...:]) и «кошачьи ушки» (^~)) и др.
- Язык продолжает активно развиваться
- Сохраняет обратную совместимость

Универсальность паттернов проектирования

- Многие темы курса применимы не только к C++
- Паттерны проектирования — универсальные решения типовых задач
 - Singleton, Factory, Observer, Strategy и другие
 - Могут быть реализованы на любом ООП языке
 - Отличаются только детали реализации
- Принципы проектирования также универсальны:
 - Принципы SOLID
 - DRY (Don't Repeat Yourself)
 - KISS (Keep It Simple, Stupid)
- Навыки проектирования переносимы между языками

Зачем нужен стиль кодирования (стайл-гайд)?

Стиль кодирования: зачем нужны стайл-гайды

- Стайл-гайд (style guide) — набор правил и соглашений по оформлению кода
- Зачем нужны стайл-гайды:
 - Повышают читаемость кода
 - Упрощают совместную работу над проектом
 - Снижают когнитивную нагрузку при чтении кода
 - Уменьшают количество ошибок
 - Упрощают поддержку и рефакторинг
- Основной принцип: **«Код пишется не для компьютера, а для людей»**

- Правила именования:
 - Переменных, функций, классов, констант
 - Префиксы, суффиксы, регистр букв
 - Семантика имён (что должно отражать имя)
- Форматирование кода:
 - Отступы и пробелы
 - Расположение скобок
 - Длина строк и перенос
- Организация кода:
 - Структура классов и файлов
 - Порядок объявления членов класса
 - Группировка и порядок включаемых файлов
- Рекомендации по использованию языковых конструкций

Yandex C++ Style Guide: общие принципы

- Официальный стайл-гайд для C++ проектов Яндекса
- Основные принципы:
 - Единообразие кода во всех проектах компании
 - Ориентация на императивный, объектно-ориентированный C++
 - Поддержка кроссплатформенности
 - Ограниченное использование STL и Boost
- Цитата из гайда:

«Вы пишете программу не для компьютера («Но она же работает!») и не для себя («Спросите меня, я объясню, как это работает»), а для всех разработчиков, которые будут её читать, изменять и отлаживать, и их потребности нужно уважать.»

Yandex Style Guide: именование

Прочитать этот слайд можно, но требуются дополнительные усилия:

Используется CamelCase, snake_case и не используется Локальные переменные и аргументы функции:

с маленькой буквы Функции и методы

классов: с большой буквы Члены

классов: с большой буквы имена

классов и структур: префикс T + имя с

большой буквы Интерфейсы: префикс I

Констан-

ты: ЗАГЛАВНЫМИ БУКВАМИ

```
const int MAX_SIZE = 100;
class TMyClass {public:
    TMyClass(int size): Size(size) {} int
    GetSize(){return Size;} private: int Size;
}; int GetValue(); void SetValue(int val);
int main() { TMyClass object; return 0; }
```

Yandex Style Guide: именование

- Используется CamelCase, snake_case не используется
- Локальные переменные и аргументы функции: с маленькой буквы
- Функции и методы классов: с большой буквы
- Члены классов: с большой буквы
- Имена классов и структур: префикс T + имя с большой буквы
- Интерфейсы: префикс I
- Константы: ЗАГЛАВНЫМИ_БУКВАМИ

```
const int MAX_SIZE = 100;
```

```
class MyClass {  
public:  
    MyClass(int size)  
        : Size(size)  
    {}  
  
    int GetSize() {  
        return Size;  
    }  
  
private:  
    int Size;  
};  
  
int GetValue();  
void SetValue(int val);  
  
int main() {  
    MyClass object;  
    return 0;  
}
```

Yandex Style Guide: перечисления

- Глобальные перечисления:
 - Имя: префикс E + имя с большой буквы
 - Элементы: ЗАГЛАВНЫМИ_БУКВАМИ с префиксом
- Перечисления с областью видимости (enum class) и перечисления внутри класса:
 - Имя: префикс E + имя с большой буквы
 - Элементы: в CamelCase

```
// Глобальное перечисление  
enum EFetchType {  
    FT_SIMPLE,  
    FT_RELFORM_DEBUG,  
    FT_ATR_DEBUG  
};
```

```
// enum class  
enum class EStatus {  
    Created,  
    Running,  
    Finished  
};
```

```
// Перечисление в классе  
class TIndicator {  
public:  
    enum EStatus {  
        Created,  
        Running,  
        Finished  
    };  
};
```

Yandex Style Guide: форматирование

- Отступы: 4 пробела (не табуляции)
- Стилль блоков: 1TBS (K&R)
- Пробелы вокруг операторов
- Нет пробелов после имени функции
- Нет пробелов в конце строки
- Перенос строки (`\n`) в конце файла

```
if (something) { // K&R стиль
    One();
    Two();
} else {
    Three();
    Four();
}

for (int i = 0; i < N; ++i) {
    // do something...
}

// Многострочные условия
if (a && b && c &&
    d && e)
{
    Op();
}
```

Yandex Style Guide: организация класса

- Метки области видимости начинаются с той же колонки, что и объявление класса
- Обязательное указание областей видимости
- Члены и методы не могут быть в одной секции
- Порядок секций:
 - Сначала public методы
 - Затем protected и private методы
- Секции с членами класса могут быть либо в начале, либо в конце описания класса (но единообразно во всём проекте!)

```
class TMyClass {  
public:  
    TMyClass();  
    ~TMyClass();  
  
    void SetMember(int value);  
    int GetMember() const;  
  
private:  
    void Initialize();  
    void Cleanup();  
  
private:  
    int Member;  
    double Value;  
};
```

Yandex Style Guide: списки инициализации

- Каждый инициализируемый член с новой строки
- Двоеточие после имени конструктора
- Запятая в начале каждой новой строки
- Отступ для всех инициализируемых членов

```
TClass::TClass()  
    : FieldA(1)  
    , FieldB("value")  
    , FieldC(true)  
{  
    // дополнительный код  
    // инициализации  
}  
  
// Альтернативный вариант  
struct T {  
    int X = 0;  
    double Y = 1.0;  
};
```

Yandex Style Guide: включаемые файлы

- Порядок включения файлов:

- 1 Парный заголовочный файл
- 2 Файлы из локальной директории
- 3 Файлы других библиотек проекта
- 4 Заголовочные файлы C

- Локальные файлы в кавычках, остальные в угловых скобках

```
// Парный заголовочный файл  
#include "some_class.h" cpp
```

```
// Файлы из локальной директории  
#include "other_class.h"
```

```
// Файлы библиотек  
#include <library/json/json_reader.h>
```

```
// Заголовочные файлы C  
#include <cmath>  
#include <cstdio>
```

Преимущества следования стайл-гайду

- Единообразие кода во всём проекте
- Снижение когнитивной нагрузки при чтении кода
- Упрощение процесса code review
- Более быстрое понимание кода новыми участниками проекта
- Возможность автоматизации проверки стиля и форматирования кода (но автоформаттер может исправить далеко не всё!)
- Повышение качества кода в целом

«Хороший стиль кода — это не просто эстетика, это инструмент коммуникации между разработчиками, который делает совместную работу более эффективной.»

Напоминание про STL

STL: что это такое

STL (Standard Template Library) — библиотека обобщённых контейнеров и алгоритмов, разработанная Александром Степановым (HP, начало 1990-х) и вошедшая в стандарт C++98

- Сегодня STL — часть стандартной библиотеки C++ (наряду с потоками ввода-вывода, строками, умными указателями и т. д.)
- В основе лежит **обобщённое программирование**: алгоритм пишется один раз и работает с любым типом, который удовлетворяет заданным требованиям
- Требования формулируются не через наследование от общего базового класса, а через набор поддерживаемых операций
- Всё реализовано на шаблонах и инстанцируется под конкретный тип на этапе компиляции:
 - Нет виртуальных вызовов и приведений типов
 - По производительности код сравним с написанным вручную

- **Контейнеры** — хранят элементы (`vector`, `map`, `set`, ...)
- **Итераторы** — дают единообразный доступ к элементам
- **Алгоритмы** — операции над диапазонами элементов (`sort`, `find`, ...)
- **Функторы и предикаты** — настраивают поведение алгоритмов
- **Аллокатеры** — управляют выделением памяти для контейнеров

Ключевая идея:

- Алгоритмы отделены от структур данных и общаются с ними **только через итераторы**
- Алгоритму не нужно знать, с каким контейнером он работает
- M алгоритмов и N контейнеров требуют **$M + N$** реализаций вместо $M \times N$

Итераторы — связующее звено

Итератор — обобщение указателя: поддерживает разыменование (`*it`), переход к следующему элементу (`++it`) и сравнение (`it != end`)

Диапазон задаётся **полуинтервалом** `[begin, end)`: `end` указывает за последний элемент. Тогда пустой диапазон — это просто `begin == end`.

```
std::vector<int> v = {3, 1, 2};
```

```
for (auto it = v.begin(); it != v.end(); ++it) {  
    std::cout << *it;  
}
```

```
for (const auto& x : v) { // то же самое, C++11  
    std::cout << x;  
}
```

```
int arr[3] = {3, 1, 2}; // работаем и с C-массивом  
std::sort(std::begin(arr), std::end(arr));
```

Категории итераторов

Чем больше умеет итератор, тем больше алгоритмов ему доступно:

- ❶ **Input / Output** — однократные, только чтение или только запись (например, потоковые итераторы)
- ❷ **Forward** — многократные, только вперёд (`forward_list`, `unordered_*`)
- ❸ **Bidirectional** — вперёд и назад, есть `--it` (`list`, `map`, `set`)
- ❹ **Random access** — переход на произвольное число шагов за $O(1)$: `it + n`, `it[n]` (`vector`, `deque`, `array`)
- ❺ **Contiguous** (C++17/20) — random access плюс гарантия непрерывности памяти (`vector`, `array`, `string`)

Практические следствия:

- `std::sort` требует random access, поэтому не работает с `std::list` — у списка есть собственный метод `sort()`
- `std::advance(it, n)` для `vector` стоит $O(1)$, а для `list` — $O(n)$

Обзор стандартных контейнеров

- **Последовательные:**

- `std::array` — массив фиксированного размера
- `std::vector` — динамический массив
- `std::deque` — двусторонняя очередь
- `std::list`, `std::forward_list` — дву- и односвязный списки

- **Упорядоченные ассоциативные (красно-чёрное дерево):**

- `std::map`, `std::set`, `std::multimap`, `std::multiset`

- **Неупорядоченные ассоциативные (хеш-таблица):**

- `std::unordered_map`, `std::unordered_set` и их multi-варианты

- **Адаптеры (обёртки над контейнерами):**

- `std::stack`, `std::queue`, `std::priority_queue`

Особняком стоит `std::string`: формально не контейнер, но поддерживает почти весь его интерфейс

std::vector: внутреннее устройство

Внутри вектора — три указателя на непрерывный блок динамической памяти:

```
begin           end           end_of_capacity
|              |              |
v              v              v
+---+---+---+---+---+---+---+---+---+---+
| 1 | 2 | 3 |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+---+---+
\ size() = 3 /
\_____ capacity() = 10 _____/
```

- `size()` — сколько элементов есть сейчас
- `capacity()` — сколько элементов поместится без перевыделения памяти
- Элементы лежат **подряд** $\Rightarrow$ доступ по индексу за $O(1)$ и дружелюбность к кэшу процессора
- `data()` возвращает указатель на буфер — вектор можно передать в C API

Что происходит при `push_back`, когда `size() == capacity()`:

- 1 Выделяется новый буфер большего размера (обычно в 2 раза в GCC и Clang, в 1.5 раза в MSVC)
 - 2 Элементы перемещаются (или копируются) в новый буфер
 - 3 Старый буфер освобождается
- Рост в константное число раз даёт **амортизированную $O(1)$** вставку в конец
 - При реаллокации **инвалидируются все итераторы, указатели и ссылки** на элементы
 - `reserve(n)` заранее выделяет память и убирает лишние реаллокации
 - `shrink_to_fit()` — необязательный к исполнению запрос вернуть лишнюю память
 - Вставка и удаление в середине — $O(n)$: элементы сдвигаются

std::vector: реаллокация на практике

```
std::vector<int> v;
for (int i = 0; i < 10; ++i) {
    v.push_back(i);
    std::cout << v.size() << " " << v.capacity() << "\n";
}
// size/capacity: 1/1  2/2  3/4  4/4  5/8  ...  9/16  10/16
// 4 реаллокации на 10 вставок
```

```
std::vector<int> u;
u.reserve(10);      // одно выделение памяти
for (int i = 0; i < 10; ++i) {
    u.push_back(i); // реаллокаций не будет
}
```

- Если число элементов известно хотя бы примерно — вызывайте `reserve`
- `emplace_back` конструирует элемент прямо в буфере, без временного объекта (подробнее — в лекции про семантику перемещения)

`std::array`, `std::list` и `std::forward_list`

`std::array<T, N>`

- Массив фиксированного размера, размер — часть типа
- Память не в куче (обычно на стеке), нулевые накладные расходы
- В отличие от C-массива, знает свой размер и не распадается в указатель

`std::list<T>` — двусвязный список

- Вставка и удаление по итератору за $O(1)$, `splice` переносит элементы без копирования
- Итераторы и ссылки не инвалидируются при операциях с другими элементами
- Но: отдельный узел и отдельная аллокация на каждый элемент, обход прыжками по памяти
- На практике почти всегда проигрывает `std::vector` из-за промахов кэша

`std::forward_list<T>` — односвязный список

- Экономит указатель на узел; отсюда `insert_after`, `before_begin()` и отсутствие `size()`

std::deque: внутреннее устройство

Двухуровневая структура: непрерывный массив указателей на блоки, в блоках лежат сами элементы



- Размер блока B фиксирован и задан реализацией (в `libstdc++` — 512 байт, но не меньше одного элемента)
- Доступ по индексу за $O(1)$: номер блока — это i / B , смещение внутри блока — остаток от деления. Но двойная косвенность делает его медленнее, чем у вектора
- Память **не непрерывна**: у `deque` нет `data()`, его нельзя передать в C API
- Вставка в середину — $O(n)$

std::deque: почему вставка в конец — $O(1)$

Три случая при `push_back` (для `push_front` всё симметрично):

- ❶ **Обычный случай:** в последнем блоке есть свободное место — элемент конструируется прямо в нём. Ни аллокаций, ни копирований
 - ❷ **Последний блок заполнен:** выделяется **один** новый блок и его адрес записывается в свободную ячейку массива указателей. Случается раз в B вставок
 - ❸ **В массиве указателей кончились свободные ячейки:** он перевыделяется — либо указатели просто сдвигаются к центру, если запас остался с другой стороны. Копируются **только указатели**, их n/B штук
-
- Третий случай редкий и стоит $O(n/B) \Rightarrow$ вставка в конец — **амортизированная $O(1)$** , как и у вектора
 - Но, в отличие от вектора, **сами элементы не переезжают никогда** — переносятся только указатели на блоки
 - Поэтому указатели и ссылки на элементы остаются валидными
 - А итераторы — нет: итератор хранит указатель на ячейку массива указателей, а сам этот массив мог переехать

`std::map` и `std::set`: красно-чёрное дерево

Внутри — **красно-чёрное дерево**: двоичное дерево поиска, которое само себя балансирует

- Каждый узел покрашен в красный или чёрный цвет, корень и пустые поддеревья — чёрные
- У красного узла оба потомка чёрные
- На всех путях от узла вниз одинаковое число чёрных узлов

Эти инварианты гарантируют высоту $O(\log n)$; при вставке и удалении они восстанавливаются поворотами и перекрашиванием

Следствия для `map` и `set`:

- Поиск, вставка и удаление — **гарантированные** $O(\log n)$, плохих случаев нет
- Обход контейнера идёт **в отсортированном порядке**
- Ключ должен поддерживать `operator<` или свой компаратор
- Элементы лежат в отдельных узлах $\Rightarrow$ итераторы и ссылки стабильны, но есть накладные расходы (указатели и цвет) и промахи кэша
- `multimap` и `multiset` допускают одинаковые ключи

std::map: что стоит помнить

```
std::map<std::string, int> ages;

ages["Ivan"] = 30;           // вставка

int x = ages["Petr"];        // создаст элемент Petr
                             // со значением 0 и вернёт ссылку
int y = ages.at("Petr");     // бросит исключение, если нет

if (ages.count("Petr")) {}   // до C++20
if (ages.contains("Petr")) {} // C++20

auto it = ages.find("Ivan");
if (it != ages.end()) {
    it->second += 1;
}

for (const auto& [name, age] : ages) { // в порядке имён
    std::cout << name << ": " << age << "\n";
}

auto lo = ages.lower_bound("A"); // диапазонные запросы
```

`std::unordered_map` и `std::unordered_set`: хеш-таблица

Внутри — **хеш-таблица**: массив бакетов, номер бакета вычисляется из хеша ключа

- Элементы, попавшие в один бакет (коллизии), хранятся вместе — обычно связным списком (метод цепочек)
- От ключа требуются `std::hash<Key>` и `operator==` (а не `operator<`, как для `map`)
- Поиск, вставка и удаление — $O(1)$ **в среднем** и $O(n)$ в худшем случае, когда все ключи попали в один бакет
- Когда коэффициент загрузки (`load_factor`) превышает порог, происходит **rehash**: бакетов становится больше, хеши всех элементов пересчитываются
- `reserve(n)` заранее готовит таблицу под n элементов и убирает лишние rehash
- Порядок обхода не определён и меняется после rehash
- Для своих типов нужна специализация `std::hash` или свой функтор в параметрах шаблона

Подробнее хеш-таблицы разберём в отдельной лекции курса

std::map vs std::unordered_map

std::map (дерево):

- Гарантированные $O(\log n)$ в любом случае
- Обход в отсортированном порядке, работают `lower_bound` и `upper_bound`
- Нужен только `operator<`

std::unordered_map (хеш-таблица):

- $O(1)$ в среднем — на больших объёмах обычно заметно быстрее
- Порядка нет, диапазонных запросов нет
- Нужны `std::hash` и `operator==`, качество хеш-функции критично

Как выбирать:

- Нужен порядок, диапазоны или предсказуемость в худшем случае — `map/set`
- Нужен только быстрый поиск по ключу — `unordered_map/unordered_set`
- На небольших объёмах (десятки элементов) отсортированный `vector` часто обгоняет оба

Адаптеры контейнеров

Адаптер — это **не контейнер**, а обёртка, которая **сужает** интерфейс базового контейнера до нужных операций

- Базовый контейнер задаётся вторым параметром шаблона:
 - `std::stack` и `std::queue` — по умолчанию над `std::deque`
 - `std::priority_queue` — по умолчанию над `std::vector`
- Своей памятью адаптер не управляет — всё делает базовый контейнер
- У адаптеров **нет итераторов**: `begin()` и `end()` отсутствуют
 - Алгоритмы STL к ним неприменимы
 - Обойти элементы, не разрушив адаптер, нельзя
- Смысл — именно в ограничении: интерфейс сам документирует, как со структурой можно работать

stack vs queue

- `std::stack` — **LIFO**: доступ только к вершине, последним пришёл — первым ушёл
- `std::queue` — **FIFO**: кладём в хвост, забираем из головы
- `std::priority_queue` — двоичная куча над вектором: `top()` даёт максимальный элемент, `push` и `pop` за $O(\log n)$

```
std::stack<int> s;  
s.push(1); s.push(2); s.push(3);  
s.top();    // 3 --- последний положенный  
s.pop();
```

```
std::queue<int> q;  
q.push(1); q.push(2); q.push(3);  
q.front(); // 1 --- первый положенный  
q.pop();
```

`pop()` ничего не возвращает: если бы он отдавал элемент по значению и копирование бросило исключение, элемент был бы уже удалён и потерян. Поэтому чтение (`top/front`) и удаление (`pop`) разделены.

Алгоритмы STL

Алгоритмы: общий принцип

Около сотни алгоритмов в `<algorithm>` и `<numeric>`. Все они работают с **диапазоном итераторов** и ничего не знают о контейнере.

```
std::vector<int> v = {3, 1, 4, 1, 5};

std::sort(v.begin(), v.end());           // 1 1 3 4 5
auto it = std::find(v.begin(), v.end(), 4);
int n = std::count(v.begin(), v.end(), 1); // 2
int sum = std::accumulate(v.begin(), v.end(), 0); // 14

std::list<int> l = {3, 1, 4};
std::find(l.begin(), l.end(), 4); // тот же алгоритм

int arr[] = {3, 1, 4};
std::find(std::begin(arr), std::end(arr), 4); // и здесь
```

- **Немодифицирующие:** `find`, `find_if`, `count`, `all_of`, `any_of`, `none_of`, `equal`, `search`, `mismatch`
- **Модифицирующие:** `copy`, `transform`, `fill`, `generate`, `replace`, `remove`, `reverse`, `rotate`, `unique`
- **Сортировка и связанное:** `sort`, `stable_sort`, `partial_sort`, `nth_element`, `is_sorted`
- **Поиск в отсортированном диапазоне:** `binary_search`, `lower_bound`, `upper_bound`, `equal_range`, `merge`
- **Операции над множествами:** `set_union`, `set_intersection`, `set_difference`
- **Работа с кучей:** `make_heap`, `push_heap`, `pop_heap`, `sort_heap`
- **Минимум и максимум:** `min`, `max`, `minmax`, `min_element`, `max_element`
- **Численные (<numeric>):** `accumulate`, `inner_product`, `partial_sum`, `iota`

Алгоритмы, предикаты и компараторы

Поведение алгоритма настраивается функтором — чаще всего лямбда-выражением

```
struct TPerson {
    std::string Name;
    int Age;
};

std::vector<TPerson> people = /* ... */;

// Компаратор: сортировка по возрасту
std::sort(people.begin(), people.end(),
    [](const TPerson& a, const TPerson& b) {
        return a.Age < b.Age;
    });

// Предикат: варианты алгоритмов с суффиксом _if
auto it = std::find_if(people.begin(), people.end(),
    [](const TPerson& p) { return p.Age >= 18; });

bool allAdults = std::all_of(people.begin(), people.end(),
    [](const TPerson& p) { return p.Age >= 18; });
```

Ловушка: алгоритмы не меняют размер контейнера

Алгоритм знает только итераторы — он не может ни удалить элемент из контейнера, ни добавить в него новый

```
std::vector<int> v = {1, 2, 3, 2, 4};
```

```
// НЕПРАВИЛЬНО: remove ничего не удаляет
```

```
std::remove(v.begin(), v.end(), 2);
```

```
// v == {1, 3, 4, ?, ?}, v.size() по-прежнему 5
```

```
// remove сдвигает нужные элементы в начало и возвращает
```

```
// новый логический конец диапазона
```

```
// ПРАВИЛЬНО: идиома erase-remove
```

```
v.erase(std::remove(v.begin(), v.end(), 2), v.end());
```

```
// v == {1, 3, 4}
```

```
// C++20: то же самое одной строкой
```

```
std::erase(v, 2);
```

```
std::erase_if(v, [](int x) { return x % 2 == 0; });
```

Итоги: алгоритмы и выбор контейнера

Почему алгоритм лучше рукописного цикла:

- Имя алгоритма сразу говорит о намерении
- Нет типичных ошибок с границами диапазона
- Реализация в стандартной библиотеке хорошо оптимизирована
- C++17: политики выполнения (`std::execution::par`) — распараллеливание одним аргументом
- C++20: `std::ranges` — `std::ranges::sort(v)` вместо пары итераторов, плюс ленивые представления

Как выбирать контейнер:

- По умолчанию — `std::vector`, и почти всегда с `reserve`
- Нужны вставки с обоих концов — `std::deque`
- Нужно ограничить интерфейс структуры — адаптеры `stack/queue`
- `std::list` — редко: ради стабильности итераторов и `splice`
- Нужен только быстрый поиск по ключу — `std::unordered_map/std::unordered_set`
- Нужен порядок или диапазонные запросы — `std::map/std::set`